# Finding and Exploiting Discrepancies in QR Code Payload Processing

Jianyu Lu
The Chinese University of Hong Kong
Hong Kong SAR, China
zeddyu.lu@link.cuhk.edu.hk

Yikang Chen
The Chinese University of Hong Kong
Hong Kong SAR, China
cy021@ie.cuhk.edu.hk

Wong Kin Ting
The Chinese University of Hong Kong
Hong Kong SAR, China
1155144252@link.cuhk.edu.hk

Xianbo Wang
The Chinese University of Hong Kong
Hong Kong SAR, China
xianbo@ie.cuhk.edu.hk

Che Wei Tu
The Chinese University of Hong Kong
Hong Kong SAR, China
tonytu1999@link.cuhk.edu.hk

Mak Sui Ling Angela
The Chinese University of Hong Kong
Hong Kong SAR, China
angelalamak@link.cuhk.edu.hk

Sze Yiu Chau
Simon Fraser University
Burnaby, British Columbia, Canada
sychau@sfu.ca

## Abstract

QR codes are now widely integrated into mobile apps as a means of contactless information dissemination. By simply scanning a QR code, many actions, such as opening a website or making a payment, can be simplified and require fewer user inputs, improving convenience and user experience. Nevertheless, apps from different vendors may not use the same decoding and payload-processing logic, creating hidden discrepancies in practice.

In this paper, we systematically analyze these discrepancies and discuss scenarios in which they can enable deception. By crafting payload data that exercises differences in how apps normalize, sanitize, render, and consume payload strings, an attacker may create a display–execution gap that users struggle to discern. To discover these discrepancies, we compiled 126 meaningful test cases and developed a semi-automated Android test framework. We identified in-app, inter-app, and inter-OS discrepancies that may facilitate phishing scenarios. Notable findings include misleading URL displays in popular browsers and a secure QR code scanner that can mark an unsafe URL as safe, both of which can create a false sense of security. Finally, we present root causes for selected discrepancies and discuss possible mitigations.

## CCS Concepts

• **Security and privacy** → **Software and application security**; • **Information systems** → *World Wide Web*; • **Networks** → *Application layer protocols*; • **Applied computing** → *Electronic commerce*.

## Keywords

QR Code, Mobile Security, Parsing Discrepancies, URLs



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
*CCS '26, The Hague, Netherlands*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2871-6/2026/11
https://doi.org/10.1145/3830454.3832645

**ACM Reference Format:**
Jianyu Lu, Yikang Chen, Wong Kin Ting, Xianbo Wang, Che Wei Tu, Mak Sui Ling Angela, and Sze Yiu Chau. 2026. Finding and Exploiting Discrepancies in QR Code Payload Processing. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26), November 15–19, 2026, The Hague, Netherlands.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3830454.3832645

## 1 Introduction

A Quick Response (QR) code is a type of two-dimensional barcode that allows devices with cameras to obtain data in a contactless manner. Besides Web-based marketing campaigns and food ordering, QR codes have also seen increased use in banking apps, as a convenient way of sending monetary transactions to the intended recipient. Given the prominence of QR codes, in this paper, we seek to investigate the discrepancies in QR code payload processing that might be exploitable. As shown in Figure 1, the consumption of a QR code can be broken into three stages: decoding (stage 1), optional parsing, validation and rendering (stage 2), and execution (stage 3). Existing works studied bugs and vulnerabilities in stages 1 and 3 [8, 23, 28, 57, 58, 63], and we expand on the literature to consider exploitable discrepancies in payload parsing and display (stage 2).

Although it is unsurprising that QR codes can be used to carry phishing URLs [28], by taking advantage of payload processing discrepancies, it is possible to achieve stealthy deceit. The first line of defense against QR-code based phishing attacks often relies on human decision making. Users typically encounter QR codes through two pathways. In the first case, they scan using a general-purpose scanner such as a system camera application or secure QR code scanner, which then passes the decoded payload to the target application for execution. In the second case, users scan directly within the target application itself, such as a browser's built-in QR code scanner. Regardless of the pathway, many applications implement stage 2 to display the decoded payload and request user confirmation before proceeding to execution, whereas others navigate directly and leave the browser address bar as the first opportunity

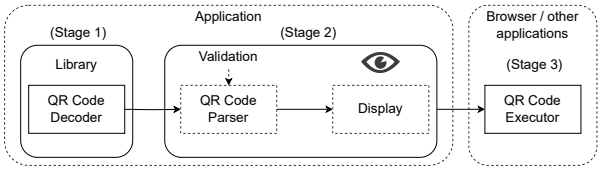



**Figure 1: The three phases of decoding a QR code**

for users to inspect the destination. These user-facing interfaces enable users to inspect the destination and make informed decisions about whether to proceed.

The importance of stage 2 is widely recognized in the security community. The absence of this confirmation step in older versions of Brave and Firefox was severe enough to warrant CVE assignments (CVE-2023-28364 and CVE-2024-0953) [13, 14]. Without stage 2, applications proceed directly from decoding to execution, removing a pre-navigation opportunity for users to detect and stop malicious payloads. However, by leveraging the discrepancies between visuals and execution semantics discussed in this paper, an attacker can make the phishing attack difficult to detect, even for a security-conscious user.

To uncover QR code payload processing discrepancies across various applications and platforms, we conduct a systematic study of user-facing apps on Android and iOS, the two most popular mobile platforms to date. For Android, we designed and built a semi-automated testing framework. Using this framework, we can obtain the intermediate results from each of the 3 stages of QR code consumption. Given the vast input space and that visual deceits are difficult to model with typical coverage-based metrics, mutation-based fuzzing is unlikely to be effective in finding exploitable display discrepancies. Instead, we draw inspiration from previous work on web-link attacks [2, 4, 10, 19, 21, 22, 26, 30, 32–34, 41, 43, 47, 50, 56] and prepared a collection of 126 test cases for targeted testing. Using these, we are able to discover numerous *in-app*, *inter-app*, and *inter-OS* discrepancies. Notably, we found many exploitable discrepancies in popular browsers including Firefox, Chrome, and Edge, as well as in secure QR code scanners from anti-virus vendors intended to help users avoid malicious websites. Many of these stem from divergent tactics embraced by developers in normalizing and sanitizing the payload string. Interestingly, discrepancies can occur across Android and iOS even for the same app from the same developer. Taking advantage of this, attackers can craft a QR code that induces apps on different platforms to load different web links, enabling targeted attacks. This could have security implications for peer-to-peer payment service apps.

Our findings have been responsibly disclosed to the corresponding vendors, with several fixes already being planned, and we received a bug bounty from Google for the Chrome display defects. Overall, the results of this study show that QR code payload processing is fragmented and can offer users inconsistent experiences. Specifically, many user-facing apps do not provide adequate support for users to discern potential phishing web links, enabling deception in several application scenarios. This status quo can be partly attributed to heterogeneous developer choices and the lack of a standard specification for processing QR code payload data that contains URLs.

**Contributions.** In short, this paper makes the following technical contributions:

1. We analyze the scanning processes in QR code applications and categorize different types of payload processing and rendering discrepancies.
2. To uncover discrepancies that may enable deceptive attacks, we conduct targeted tests of the scanning workflow, covering popular mobile browsers and QR code scanners.
3. For testing Android apps, we build a semi-automated framework that helps identify exploitable discrepancies and triage their root causes.
4. We dissect the observed discrepancies and discuss possible exploitation and mitigation.

## 2 Background

### 2.1 QR Code construction

In general, a QR code includes an encoding area with data and error-correcting codewords, as well as function patterns that hold symbol metadata and positioning information. Following the ISO/IEC 18004:2015 [1] standard, the steps needed to construct a QR code are outlined below.

**Data Analysis:** First, the optimal encoding mode is determined for the text string to be converted into a QR code. Choices include *numeric* mode for encoding numbers, *alphanumeric* mode for alphanumeric characters, *byte* mode for binary data and *kanji* mode for Japanese characters.

**Data Encoding:** The encoder transforms the text string into a sequence of bytes using the selected encoding mode.

**Error Correction Encoding:** The Reed-Solomon algorithm [62] is applied to generate error correction codewords. These codewords provide redundancy and are added to the original message to enable error detection and correction during QR code decoding.

**Final Message Structuring:** Data and error correction codewords are combined and arranged in a specific order to form the final message to be inserted into the QR code.

**Module Placement in Matrix:** The resulting message bits are laid out in the module matrix (pixels) that form the QR code pattern. This pattern uses interleaving to enhance error resistance and correction capability, and it varies according to the QR code version.

**Data Masking:** To improve readability and error detection capability, a masking pattern is applied to the data matrix. This pattern alters bits in the matrix according to a specific masking formula.

**Format and Version Information:** Finally, format and version information is added to the QR code. Format information includes details about data masking.

### 2.2 URL

A URL is a reference for locating and accessing resources on the World Wide Web, following the standards set by the Internet Engineering Task Force (IETF) [3] and the Web Hypertext Application Technology Working Group (WHATWG) [60]. Most browsers follow the WHATWG standards to parse URLs. A URL usually consists of several key components:

**Scheme**: Indicates the protocol (e.g., `http:\\`, `https:\\`).

**Authority**: Includes the domain name or IP address and an optional port number (e.g., `www.example.com` or `127.0.0.1:8080`). Optional credentials can also be included (*e.g.*, `user:pwd@example.com`).
**Path**: Specifies the resource's relative location on the server (e.g., `/path/to/resource`).
**Query**: Provides additional parameters, starting with a question mark and consisting of key-value pairs (e.g., `?param1=value1&param2=value2`).
**Fragment**: Identifies a specific part of the resource, starting with a hash (e.g., `#section1`).

## 2.3 Unicode

Applications that process QR codes often find the need to render Unicode text extracted from their payload data. Interestingly, text rendering itself, if not performed with care, can open doors to deceptive attacks [5, 6], especially when the text is used as URLs [11, 12]. From the literature, several categories of characters can exploit this attack vector:
**Invisible characters**, also known as non-printing or control characters, are a subset of characters in the Unicode standard that do not produce a visible mark when rendered. They are useful in controlling how text is interpreted and displayed.
**Homographs** are characters that appear similar to each other but have different Unicode code points. These characters can come from different scripts (such as Latin, Cyrillic, or Greek) or within the same script, causing visual ambiguity. For instance, the Latin letter "A" and the Cyrillic letter "A" look almost identical in Unicode.
**Reordering characters** in Unicode can rearrange characters' visual presentation relative to their logical order, which is especially relevant in scripts with complex text layouts, such as Arabic and Hebrew. These scripts require bidirectional text algorithms to display mixed-direction text correctly. While reordering ensures accurate representation of text in various languages, it also introduces potential visual confusions to end-users.

# 3 Discrepancies and Deceptive Potential

## 3.1 Types of Discrepancies

**In-App Discrepancies.** The first type of discrepancy that we consider focuses on the gap between the UI display shown to the user (stage 2) and the effective payload data being acted upon (stage 3), all *within the same application*. Imagine a browser that, upon extracting data from a scanned QR code, shows the URL to the user in a pop-up notification before the user agrees to actually loading the Website. If the notification display fails to faithfully communicate the effective URL due to careless handling and rendering of the special characters discussed in Section 2.3, the users could be tricked into loading a malicious Website.
**Inter-App Discrepancies.** Next, we consider the data validation and processing logic implemented in *different apps* that perform similar functionalities. While these apps may offer comparable features and UI designs on the surface, the ways in which they sanitize and interpret the payload data from the same QR code (stages 2 and 3) can differ significantly, in part due to different programming conventions and the use of heterogeneous libraries and APIs. Such differences can be exploited to make groups of users experience different execution outcomes, allowing an attacker to perform covert but targeted deceit against a specific user group.
**Inter-OS Discrepancies.** Beyond individual applications, differences in programming and interpretation conventions also extend to the underlying *operating systems* (OSes). Thus, we also consider the system APIs and default apps that are found on mainstream mobile platforms. Discrepancies can exist in the same app (e.g., Firefox) that runs on different platforms (e.g., Android and iOS), as well as the various system default camera apps shipped by different vendors (e.g., Apple, Google, Samsung, etc.). These can be similarly exploited by making different user groups experience different execution outcomes when scanning the same QR code.

The three discrepancy categories are instances of the same requirement for payload consistency. For a QR code that contains a URL, the destination represented to the user, examined by a security mechanism, and passed to an executor should identify the same initial origin. This requirement applies to the complete processing workflow rather than to an individual parser or application. A discrepancy at any processing boundary can break this consistency even when the individual components process their inputs as intended.

## 3.2 Deceptive Attacks and Threat Model

We now briefly discuss why discrepancies in the processing of QR code payload can be exploited by an attacker to launch deceptive attacks. We assume the attacker can provide maliciously crafted QR codes and induce users to scan them. This can happen by replacing or patching over a benign QR code, especially in settings where QR-code-containing banners and business cards are *printed by a third party*, or the printed copies of a benign QR code are *physically accessible to the general public* in restaurants and bus stops. Because a malicious QR code can appear in a seemingly legitimate context (e.g., business cards and advertising campaigns), users may be inclined to scan it. The goal of the attacker then is to get the users to agree to load a malicious website in certain mobile apps, especially web browsers. This is because a malicious website can be designed to leverage a variety of other web attack vectors, such as cross-site scripting (XSS) [45], browser exploits [31], or cryptojacking [61].
**Main advantage over conventional attacks.** To defend against QR-code based phishing, some scanning apps implement a mechanism that displays a confirmation dialog during stage 2 of its processing pipeline. For example, both Firefox [14] and Brave [13] have implemented a confirmation dialog to obtain explicit user consent before loading Websites from scanned URLs. Additionally, some secure QR code scanners might attempt to detect and warn users of known malicious URLs from blocklists. However, these mechanisms might be rendered ineffective by someone who exploits discrepancies in the display and execution semantics. For instance, an attacker can construct a QR code that, when scanned by the user, causes the app to display a URL that appears legitimate (e.g., `apple.com`), but actually opens a website on a different domain (e.g., `attacker.com`)[1]. In such cases, even cautious users may be unable to identify the deceptive destination because the display fails to communicate the effective URL that is loaded.

[1]A 17-second demonstration video with Chrome on iOS can be found at https://youtu.be/Naq5a1V-ipI

# 4 Methodology

## 4.1 Test subject selection

A full list of the selected test subjects and their versions is provided in Table 1. For in-app and inter-app discrepancies, we focus on popular browser apps, some prevalent QR code readers and two secure QR code scanners on Android and iOS. This is primarily due to three reasons. First, these apps have inbuilt QR code scanning functionalities by design. Second, many live-service apps require user registration and/or identity verification, but browsers and QR code readers and are typically immediately usable after installation, making testing much easier. Third, browsers and generic QR code scanners serve as users' *first line of defense* against malicious URLs. Unlike live-service applications that typically restrict URL processing to specific domains, browsers and QR code scanners are designed to handle URLs of any domain. This universal processing capability places them at the forefront of phishing prevention, making their display mechanisms particularly critical for helping users identify potential threats. For inter-OS discrepancies, with the mobile devices available in our lab, we consider the system default apps for scanning QR code on iOS (Apple Camera) , stock Android and some branded Android (*i.e.*, the default camera apps from Samsung and Sony).

| App | OS | Version |
|---|---|---|
| Brave | iOS 17.5.1(21F90) | N/A[1] |
| Chrome | | 128.0.6613 |
| Edge | | 127.0.2651 |
| Firefox | | 129.1 |
| Kasperky | | 2.95.0.29 |
| Opera | | 5.0.5 |
| ScanDemo | | N/A[2] |
| Trend Micro Mobile Security | | 12.0.1090 |
| UC Browser | | 17.0.3.2425 |
| Vivaldi | | 6.8.3388 |
| iOS 17 Camera | | N/A[1] |
| QR Code Reader ϟ | | 9.5 |
| QR Code Reader & Scanner App - | | 3.4.1 |
| QR Code Reader · | | 2.4.14 |
| QR Code & Barcode Scanner · | | 2.5.4 |
| QR Code Reader & QR Scanner | | 1.97 |
| Brave | Android 13 | 1.69.153 |
| Chrome | | 127.0.6533 |
| Edge | | 127.0.2651 |
| Firefox | | 129.0.2 |
| Kaspersky Free | | 11.118.4 |
| Opera | | 84.2.4452 |
| Pixel Camera | | N/A |
| Trend Micro QR Scanner | | 1.2.1.0079 |
| UC Browser | | 13.7.8 |
| Vivaldi | | 6.8.3388 |
| QR & Barcode Reader | | 1.0.26 |
| QR & Barcode Scanner | | 2.2.59 |
| QR Code Scanner | | 1.3.16 |
| QR Scanner_Barcode Scanner | | 2.6.5 |
| Samsung Camera | Android 13 One UI 5.1 | 13.1.00.62 |
| Sony Camera | Android 13; Build/63.1.B.0.561 | 2.04.096 |

[1] Not Applicable. No version for iOS Camera.
[2] Not Applicable. Developed by us based on the Apple QR code API

**Table 1: Tested apps and OS versions**

## 4.2 Testing for Discrepancies and Deception

For in-app, inter-app, and inter-OS discrepancies, we perform *end-to-end testing*. That is, for each test case, we generate a QR code and let the apps scan, process, and execute actions based on its payload data.

**Test case preparation.** As outlined in Section 3.1, we are interested in whether special characters in QR code payload data can induce in-app and inter-app discrepancies. For this, we surveyed existing literature for characters that can be useful in deceits and are thus meaningful to explore. The compiled test cases can be divided into three categories: *Encoding*, *Confusion*, and *Miscellaneous*, which are described below. Table 2 shows the types of test cases, their high-level categories, and examples under each type.

**Encoding.** This category includes UTF-8 Overlong Bytes [10], reordering characters, deletion characters [5, 6], Unicode IDNA (Homoglyph) [4, 32], Byte-order mark (BOM), invisible white space characters and invisible control characters. These characters are often only partially supported by implementations, which can lead to ambiguity in interpretation and unexpected outcomes. Thus, we prepare and utilize the encoding dataset in testing In-App, Inter-App, and Inter-OS discrepancies, as encoding-related issues remain prevalent in diverse software environments, sometimes opening doors to exploits. For example, if an application displays Internationalized Domain Names (IDNs) directly without first converting them to Punycode, users can be tricked.

**Confusion.** These test cases are mainly designed for Inter-App testing and they are primarily derived from previous fuzz testing collections for URL parsers [2, 50]. From these studies, we selected representative payloads to include in our test set. For example, the Backslash confusion (*e.g.*, `https://apple.com\@msdn.com`) may confuse URL parsers and lead them to different hosts.

**Miscellaneous.** In this category, we explore non-HTTP schemes, such as `javascript://` and `file://`. These protocols were flagged in various vulnerability reports [22, 26, 41, 43, 47] as potential vectors for Cross-Site Scripting (XSS) vulnerabilities and other unexpected behaviors. We also include some additional tests, such as domain correction tests, which stem from past bug reports [19, 21, 56] that discussed how Google Lens on certain mobile devices would automatically correct relatively uncommon domain names of a QR code payload URL to what it considers to be the correct ones. As an example, it was reported that Google Lens would change `hellonet.arte`, the real QR code payload, into `hellonet.ar`, due to `.arte` being an uncommon top-level domain.

We also note that in order to include special characters in QR code payload data, off-the-shelf QR code generation libraries do not always deliver what we expected, due to restrictions on payload data character sets as well as the same parsing and interpretation discrepancies that we wanted to discover in apps. Consequently, we implemented a custom QR code generator (following the steps in Section 2.1) that can handle our unusual test case payloads. In our experiments, we use QR code version 7, byte mode encoding, and an error correction level of L to generate our QR code test cases. A version 7 QR code can encode up to 1,800 bytes, providing ample storage space for our test cases.

**Table 2: Test cases used in the experiment**

| Categories | Detailed Types | Examples | Number of test cases |
|---|---|---|---|
| Encoding | UTF-8 Overlong Bytes | `http://apple.com\xf8\x80\x80\x80\xafhelp` | 10 |
| | Invisible White Space | `http://apple.com\xe2\x80\x8cmsdn.com` | 3 |
| | Reordering | `\xe2\x81\xa6\xe2\x80\xaemoc.elppa//:ptth\xe2\x80\xac\xe2\x81\xa9` | 3 |
| | Deletion | `http://apple.com@\x7fa.msdn.com` | 2 |
| | Unicode IDNA(Homoglyph) | `http://icoul\xea\x9d\xb1.com` | 16 |
| | Byte-order mark (BOM) | `\xfe\xff\xfe\xff\x00h\x00t\x00t\x00p\x00:\x00/\x00 /\x00a\x00p\x00p\x00l\x00e\x00.\x00c\x00o\x00m` | 7 |
| | Invisible Control | `apple.com\rmsdn.com` | 35 |
| Confusion | Hostname Confusion | `http:\\\\apple.com\\b:c\\d@msdn.com\\` | 2 |
| | Backslash confusion | `https://apple.com\@msdn.com` | 2 |
| | Slash confusion | `http://////////apple.com/` | 1 |
| | Encoding confusion | `http://\u30d2:\u30ad@\u30d2.abc.\u30cb/\u30d2` | 1 |
| Misc | Scheme Tests | `javascript://alert(document.domain)` | 7 |
| | Domain Correction Tests | `https://www.fooco.at/hello` | 3 |
| | HTTP Prefix Recognition | `www.citi\rapple.com` | 34 |

**Engineering a semi-automated detection framework on Android.** We create a semi-automated framework that reduces repetitive manual work when detecting discrepancies in Android apps. Although UI automation solutions exist (e.g., ADBGPT [18], AutoDroid [59], and DroidAgent [64]), their focus is often on improving test coverage by automating deeper and more diverse UI interactions. For our purpose, we have a very specific focus on QR code scanning, which does not warrant long sequences of UI interactions with high variety. Consequently, we found it more efficient to build a custom framework tailored for this. A key observation behind our framework is that one can use the virtual scene camera (VCam) on the Android Virtual Device (AVD) emulator environment to create a programmable test setup, where one can automatically load the next QR code test case and trigger QR code scanning on the app under test.

Ideally speaking, we would want to be able to get fine-grained information from each of the three stages of QR code process. Thus, we decompile the Android Package Kit (APK) to identify the relevant code paths and methods, which are then hooked in the Frida dynamic instrumentation toolkit [44]. Through our custom Frida script, we were able to inject additional code that dumps the initial extracted QR code data (stage 1) from the app under test. Next, in order to gauge the pre-processed displays (stage 2), when the app shows a pop-up display after the QR code is scanned, we take a screenshot of the AVD and apply a color-based segmentation approach to isolate and extract the pop-up display region. We then apply Optical Character Recognition (OCR) using PaddleOCR [46], exclusively to this region to simulate human observation, which enables the identification and analysis of the pre-processed display content. Given that our experimental data includes special Unicode characters that pose challenges for OCR, manual verification is required to identify any potential discrepancies. However, the utilization of OCR allows us to omit manual confirmation when the OCR output *aligns with* the test input, as this indicates the absence of discrepancies. Manual verification is performed only when a mismatch exists between the provided input and the OCR output. For instance, in cases of overlong truncation where the input `apple.com.longlong.evil.com` results in the OCR output `apple.com. . .`, we manually verify the display screenshot to confirm a discrepancy.

Finally, for getting the actual domain to be loaded (stage 3), we incorporate in our framework the mitmproxy [36] to capture network traffic. This allows us to monitor and record the domain visited by the browser, after the user consents to loading the URL. In-app discrepancies exist if the stage 2 results of an app differs from its stage 3 results. Inter-app and inter-OS discrepancies are found by comparing all stage 2 and 3 results. Stage 1 results help us confirm that the discrepancies are indeed caused by stage 2 or 3.

For most of the apps we tested, it was possible to take advantage of AVD snapshots to avoid repeated UI interactions. The idea is that for each app to be tested, we only need an one-off UI-navigation effort to trigger its QR code scanning function, and we take an AVD snapshot right before the app scans. Then, for each of our test cases, we can programmatically restore this snapshot and directly start scanning a QR code test case without navigating the app UI again, which speeds up the experiments. However, we note that 2 apps (Brave and Vivaldi) require the Google service framework, which blocks root access in AVD and thus breaks the Frida setup. To overcome this, we used rootAVD [42] to gain root access in an AVD with the Google service framework. However, we found that rootAVD prevents us from creating AVD snapshots. Thus, for those 2 apps, we use repeated but automated UI navigation through Appium, which records the sequence of XPath elements we tapped manually, and replays the recorded taps for each test case.

**Manual testing for the rest of the apps.** Despite success with most of the selected Android apps, our semi-automated framework does not work for some of the apps. For instance, the Google Lens Android app is heavily obfuscated and we were unable to identify the correct target method for hooking in Frida. Furthermore, due to the inherent restrictions imposed by iOS, virtual cameras are not supported in the iOS simulators of Xcode. On the other hand, jailbreaking a physical iOS device running a recent version of iOS is not known to be possible [16], and we do not have access to the virtual device platform offered by Corellium [9]. Thus, we leave the development of an iOS discrepancy detection framework for future work. In the end, we manually tested Google Lens on Android

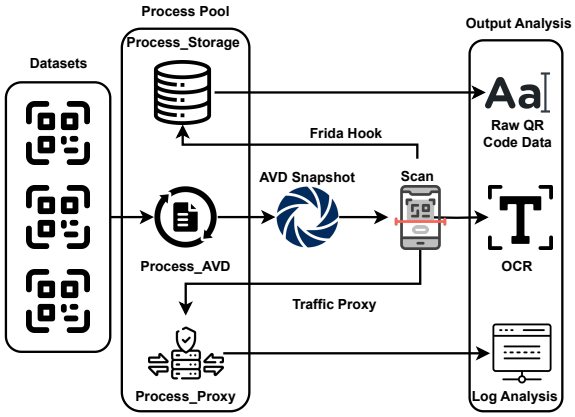



**Figure 2: Overview of our detection framework for Android**

and all the iOS apps. Although for these apps we are unable to retrieve the raw QR code data (stage 1) due to the lack of Frida instrumentation, we can still collect the stage 2 and 3 data.

**Experiment setup and time cost.** The Android experiments with our semi-automated detection framework were performed on a Mac with an 8-core Apple M2 CPU and 24 GB RAM. We found that AVD snapshot recovery is highly efficient on the ARM-based Apple M2 chip, which generally completes in under a second. This greatly improves the efficiency of our experiments for apps amenable to AVD snapshots. The emulator-based experiments for each app generally complete within hours. Manual testing of other apps took less than 1 week in total.

# 5 Findings

## 5.1 In-App Discrepancies

We categorize the attack vectors for In-App discrepancies into two categories, including Truncation and Unicode Confusion. An overview of our findings can be found in Table 3.

**Truncation.** From our analysis, we observe that truncation typically occurs in three scenarios: when URLs exceed standard length parameters, when they incorporate special Unicode characters, or when specific characters are present.

**Lengthy Truncation ($A_1$).** When the host or URL embedded within the QR code is particularly lengthy, the applications designated with $A_1$ in Table 3 may choose to display only a truncated portion of it. This hinders the user's ability to discern whether the URL data of the QR code is on a legitimate domain. Take Chrome on Android as an example, when users encounter a QR code containing the URL `http://sad.apple.com:a@msdn.com` and scan it using Chrome, the pop-up displays only `sad.apple.com...` with an ellipsis obscuring the remaining characters. This truncation deceives users into believing they are navigating to `sad.apple.com`. However, the actual destination is `msdn.com`, as the portion preceding the `@` symbol (`sad.apple.com:a`) functions as HTTP authentication credentials (username:password), not part of the host domain. Display

**Table 3: Scanning Behaviors and In-App Vulnerabilities**

| App | Platform | Behavior and In-App Discrepancies | | | | Crash |
|---|---|---|---|---|---|---|
| | | Pop-up | | Open Directly | Paste into URL Bar | |
| | | Full URL | Only Show Host | | | |
| Apple Camera | iOS | | ✓ | | | |
| Brave | iOS | | | | ✓ | |
| Chrome | iOS | ✓, $A_2$, $A_3$ | | | | |
| Edge | iOS | | | ✓, $B$ | | ✓ |
| Firefox | iOS | | ✓, $B$ | | | |
| Kaspersky | iOS | ✓, $A_1$ | | | | ✓ |
| Opera | iOS | | | ✓ | | |
| Trend Micro Mobile Security | iOS | ✓, $A_1$, $B$ | | | | ✓ |
| UC Browser | iOS | | | ✓ | | |
| Vivaldi | iOS | | | | ✓ | |
| Brave | Android | | | | ✓, $B$ | |
| Chrome | Android | ✓, $A_1$ | | | | |
| Edge | Android | ✓, $A_1$, $B$ | | | | |
| Firefox | Android | ✓, $B$ | | | | |
| Pixel Camera | Android | ✓, $A_1$, $B$ | | | | |
| Kaspersky Free | Android | ✓, $A_1$, $B$ | | | | |
| Opera | Android | | | ✓, $B$ | | |
| Trend Micro QR Scanner | Android | ✓, $A_3$, $B$ | | | | |
| UC Browser | Android | | | ✓, $B$ | | |
| Vivaldi | Android | | | ✓, $B$ | | ✓ |
| Samsung Camera | Android | ✓, $A_1$, $B$ | | | | |
| Sony Camera | Android | ✓, $A_1$, $B$ | | | | |

truncation in this case puts users in risk of phishing attacks, because even a cautious user only sees the deceptive prefix but not the effective domain.

In Pixel Camera, consecutive tab characters (`\x09`) are rendered as whitespace. When sufficiently numerous, these cause subsequent characters following the tabs to be truncated and replaced with ellipses. Conversely, Web browsers process such characters differently by omitting them entirely during URL interpretation. Specifically, given `apple.com\x09\x09\x09\x09\x09\x09\x09msdn.com`, Pixel Camera pop-up displays only `apple.com...` truncated. Yet upon confirming navigation, the browser is given the concatenated destination of `apple.commsdn.com` to load.

**Unicode Truncation ($A_2$).** Additionally, there are several other Unicode characters that exhibit similar deception potential due to truncated pop-up display. The three characters `\u3002` (Ideographic Full Stop), `\uFF0E` (Fullwidth Full Stop), and `\uFF61` (Halfwidth Ideographic Full Stop)—all functioning as Chinese period symbols—create visibility vulnerabilities in Chrome's URL rendering logic on iOS. For instance, a URL like `http://apple.com\uFF61.evil.com` appears in Chrome (iOS) pop-up notification solely as `apple.com`, while the hidden suffix transforms the actual destination into `apple.com.evil.com`.

**Specific Characters Truncation ($A_3$).** Most tested apps processed non-printable control characters (e.g., ASCII CRLF) as literals or URL-encoded data, whereas Apple Camera and Chrome (iOS) employed distinct handling mechanisms. Chrome silently truncates URLs at NULL bytes when navigating. For instance, given the payload URL `http://phishing.com\x00@hello.apple.com`, it displays `hello.apple.com` in pop-up but navigates to `phishing.com`.

Furthermore, there is another interesting case involving the handling of URLs by Chrome on iOS. When entering `http://apple.com/@msdn.com` directly, the slash before `@` correctly designates everything after as path, resolving to `apple.com`. However, scanning a QR code containing `http://apple.com\@msdn.com/` displays `apple.com/@msdn.com` in the pop-up but navigates to `msdn.com` after user confirmation. Unlike the slash, the unencoded `@` symbol (which could denote authentication credentials) is treated as a domain separator here, enabling potential URL manipulation attacks. This in-app discrepancy arises from how QR code payload data gets interpreted differently than URLs directly entered in the address bar. Figure 3 shows a test case and screenshots of this.

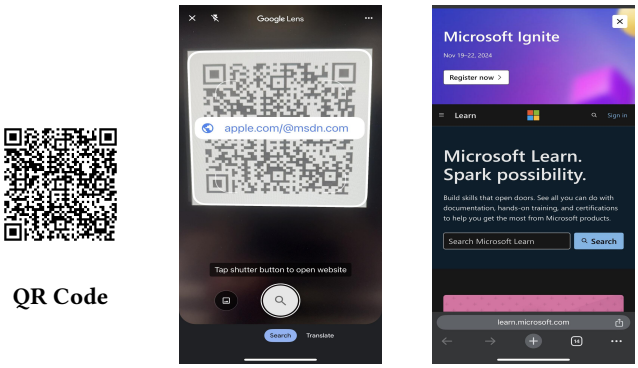



**Figure 3: Chrome on iOS scanning the QR Code with payload `http://www.apple.com\@msdn.com`**

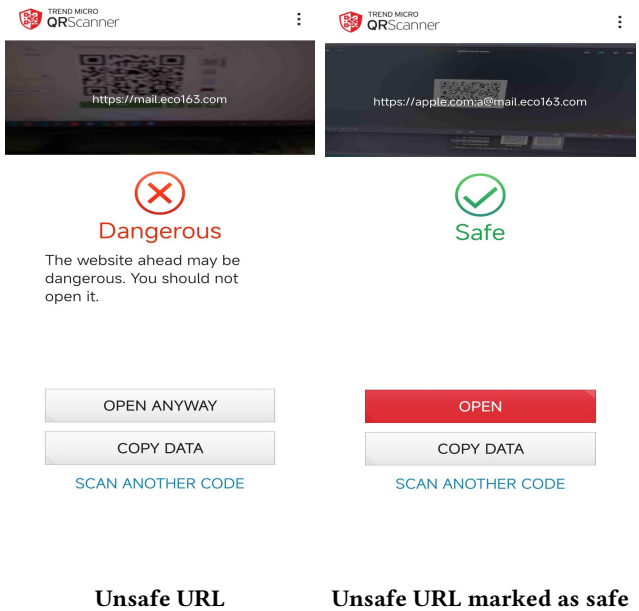



**Figure 4: Trend Micro's Secure QR Scanner**

For the secure QR code scanners, truncation techniques $A_1$ and $A_2$ did not work. For those secure QR code scanners, we cannot exploit similar length truncation techniques to conceal a malicious domain such that the scanner's verification reports the domain as safe. However, we found a case of $A_3$ that works in the Trend Micro QR Scanner on Android. When the QR code includes a URL that is originally recognized as "*Unsafe*" by Trend Micro (e.g. `http://mail.eco163.com`), we can add an HTTP credential prefix (e.g. `http://apple.com:a@mail.eco163.com`) to make Trend Micro recognize it as "*Safe*". When the user confirms opening the URL, the browser loads the "*Unsafe*" URL (`http://mail.eco163.com`). Figure 4 shows screenshots of this problem. Attackers can exploit this to masquerade malicious domains as safe, and users of this app might get a *false sense of security*.

**Unicode Confusion (*B*).** The unrestricted rendering of Unicode characters has long posed significant security concerns especially when presented in browser address bars and the URL confirmation pop-up. We categorized Unicode confusions into two distinct types: *IDN homograph attacks* where visually identical characters from different scripts (languages) create indistinguishable forged domains, and *mixed-script attacks* where multiple character sets from different languages are deliberately combined within a single URL to evade detection while maintaining visual persuasiveness. Our analysis identified vulnerabilities across multiple platforms: apps designated with a *B* in Table 3 exhibit varying degrees of susceptibility to Unicode confusions because their implementations fail to neutralize homograph or mixed-script threats. We also found that all tested QR code readers exhibit these in-app Unicode-confusion discrepancies.

**IDN Homograph attack.** For IDN Homograph attack, the deceptive homograph for `apple.com` using the URL `\u0430\u0440\u0440\u04cf\u0435.com` which consists of Cyrillic letters is a well-known attack vector. When users scan a QR code containing this URL using Firefox, Edge, Samsung and Sony Camera on Android, the pop-up prompt renders the Cyrillic letters that closely mimic `apple.com`, making the users believe that they are navigating to the legitimate site (see Figure 5 and 6 for screenshots). In reality, the actual destination is `xn--80ak6aa92e.com`[2], which could be a malicious domain under attacker's control.

Furthermore, these Unicode confusion attacks can manifest not only in pop-up interfaces but also within address bars, which is particularly critical for browsers that do not offer pop-up confirmation (*e.g.*, Opera and UC Browser), as the user will rely on this to determine the legitimacy of the Website currently being visited. In-app discrepancies between the address bar display and actual domain loaded was previously found and fixed in Chrome (CVE-2017-5060) [11] and Safari (CVE-2018-4205) [12], nevertheless, some browsers and QR code scanners continue to have similar problems. We found that there is still a gap between what is shown in the address bar and what is getting loaded in Opera and Firefox (see Figure 7 for an example with `xn--80ak6aa92e.com`[2]).

**Mixed-script attack.** Additionally, we discovered that Firefox (Android), Edge (Android), Sony Camera, and Samsung Camera are all susceptible to display rendering issues arising from Unicode

[2]This is a proof-of-concept domain maintained by Xudong Zheng, the original discoverer of CVE-2017-5060 [11]. We have no professional or personal ties with him.

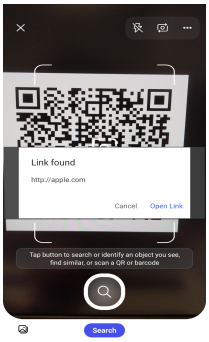



**Figure 5: Cyrillic in Edge (Android)**

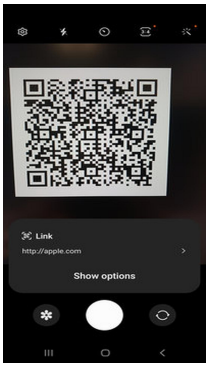



**Figure 6: Cyrillic in Samsung Camera**



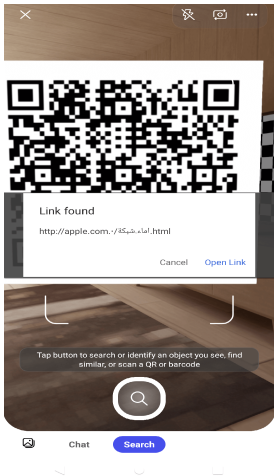

**Figure 7: IDN in Firefox address bar**

bidirectional (Bidi) interference. For instance, we can construct a QR code containing the URL `http://www.apple.com.xn--ggbla3j.xn--ngbc5azd/\u06f0.html`. In this URL, the segment `www.apple.com` is arranged in Left to Right (LTR) order, while the Punycode `xn--ggbla3j` (Unicode sequence `\u0627$\u0645\u0627\u0621`) which translates to the Arabic word "عاما", and `xn--ngbc5azd` (Unicode sequence `\u0634\u0628\u0643\u0629`) which translates to "شبكة", are presented in Right to Left (RTL) order. The final character, `\u06F0`, represents the Arabic-Indic Digit Zero, which also follows the RTL direction. This arrangement allows the URL to be displayed in the prompt as `apple.com`, creating an appearance that is highly convincing to users (see Figure 8). However, the actual destination loaded will be `xn--ggbla3j.xn--ngbc5azd`.

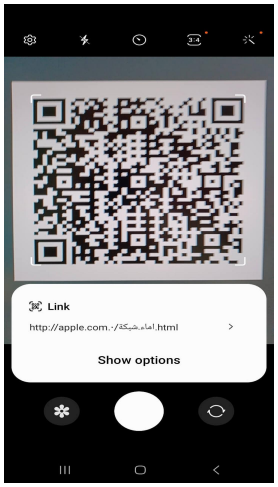
These Unicode confusions primarily stem from the programming pattern of directly displaying the QR code payload in a UI element (e.g., `TextView` in Android and `UITextView` in iOS) without proper Unicode escaping or conversion. This allows certain Unicode characters to generate misleading visual representations. Among the four secure QR code scanners, only the Kaspersky scanner on iOS displays Punycode when a QR code contains an IDN domain. The other three (Kaspersky scanner on Android and Trend Micro scanners on both Android and iOS) are susceptible to both IDN-homograph and mixed-script attacks.

## 5.2 Inter-App Discrepancies

**On iOS.** Depending on which libraries and APIs are used by the app developers, apps can have different interpretation to the same QR code payload. A prime example of this is URL handling, where various URL parsers may interpret certain characters, such as backslashes, differently, resulting in inconsistent behaviors across apps. For instance, as shown in Table 4, when given a QR code containing the payload `https://apple.com\@msdn.com`, we found that different browsers do not always navigate to the same destination. Although the backslash character is invalid in a URL according to RFC 3986, the WHATWG standard is more permissive in its handling of backslashes. As suggested by WHATWG, backslashes should be converted to slashes. Thus, given `https://apple.com\@msdn.com`, the expectation is that the browser should navigate to `apple.com` (treating the `/@msdn.com` suffix as path). However, we found that the user experience is not uniform across apps. For example, Apple Camera




**Edge**




**Samsung Camera**

**Figure 8: CVE-2018-4205-style pop-up deceptions**

**Table 4: Inter-App Handling on iOS: Backslash URL Payload**

| **Payload Data** | `https://apple.com\@msdn.com` |
|---|---|
| *Expected Result* | `msdn.com` |
| Brave† (visit) | `apple.com` |
| Chrome (display) | `apple.com//@msdn.com` |
| Chrome (visit) | `msdn.com` |
| Edge‡ (visit) | `apple.com` |
| Firefox (display) | `msdn.com` |
| Firefox (visit) | `msdn.com` |
| Opera‡ (visit) | `msdn.com` |
| UC Browser‡ (visit) | `msdn.com` |
| Vivaldi† (visit) | `apple.com` |
| iOS Camera (display) | `msdn.com` |
| iOS Safari (visit) | `msdn.com` |

†: After scanning a QR code, these browsers populate the address bar with the URL and let the user decide whether to visit it.
‡: After scanning a QR code, these browsers navigate directly to the link without displaying a prompt.

prompts the user with `msdn.com` and subsequently loads `msdn.com` in Safari, which deviates from the expectation. In contrast, Google Chrome displays `apple.com//@msdn.com` in the pop-up prompt but also navigates to `msdn.com`. Similarly, UC Browser and Opera also handle the URL by navigating to `msdn.com` (though without any pop-up prompts). On the other hand, browsers like Vivaldi and Edge interpret the URL following the WHATWG expectation; they navigate to `apple.com` while putting `apple.com/@msdn.com` in the address bar. Brave also follows this pattern, putting `apple.com//@msdn.com` in the address bar and visiting `apple.com`.

When dealing with control characters in URLs, similar discrepancies can also be observed among various browsers and applications. For example, when a QR code contains the payload `https://apple.com\nmsdn.com`, different browsers also exhibit distinct behaviors. The expected destination according to WHATWG is `apple.commsdn.com`. Firefox and Safari will attempt to visit the URL `apple.com%0Amsdn.com`, encoding the newline character. In contrast, browsers such as Edge, Brave, and Vivaldi interpret the URL following the

WHATWG expectation, treating it as a concatenated string resulting in `apple.commsdn.com`, ignoring the newline character altogether.

The situation becomes even more interesting when we consider the payload `apple.com\rmsdn.com`, which omits the `https://` protocol prefix. In this case, Apple Camera prompts the user with `apple.com` and proceeds to open it in Safari. In contrast, Google Chrome navigates to `msdn.com`, suggesting a prioritization of the second part of the URL after the carriage return character. Meanwhile, Vivaldi interprets the input as a single string and navigates to `apple.commsdn.com`, once again ignoring the separation between the two parts. Other browsers may interpret the input as plain text rather than a valid URL and query the default search engine instead. Tables 5 and 6 provide the complete browser behaviors for these two control-character payloads.

Furthermore, we also observed inconsistent behaviors in the handling of certain whitespace characters. For instance, the zero-width space is usually not rendered (and thus invisible) in the `UITextView` component. If one sets the QR code payload to `https://evil.app\u200Ble.com`, the rendered display can appear as `evil.apple.com`. However, when this gets passed to the downstream consumer app, unexpected inconsistencies can happen. For example, using the *QR Code Reader & Scanner App* - by Komorebi Inc. to scan such QR code, the reader app shows `https://evil.apple.com`. Interestingly, if users attempt to open this link with Safari, they will be directed to `evil.app`, whereas opening the same link with any other browsers will go to `evil.apple.com`.

**On Android.** Unlike iOS, the handling of backslashes is notably consistent across most browsers. For instance, given a QR code payload of `https://apple.com\@mdsn.com`, all browsers except Edge will navigate to `apple.com`, which follows the WHATWG expectation. In contrast, Edge interprets the payload with backslash as text, and uses a search engine to look up the entire text.

Nevertheless, discrepancies can also arise in the treatment of other control characters. For example, when the URL `https://apple.com\x00@mdsn.com` is included in a QR code, the behavior varies significantly across different browsers. In this case, Opera navigates to `apple.com`, while Edge defaults to using the standard search engine to search the text. The remaining browsers, however, interpret the input differently and navigate to `msdn.com`, highlighting the inconsistencies in how control characters are processed.

We also found that the decoding methodology employed by Pixel Camera differs from the conventional norm, as it performs URL decoding before display, causing parsing anomalies in URL-encoded credentials. For instance, when scanning `https://apple.com:443%2F@msdn.com`, its preprocessing resolves `%2F` to /, normalizing the URL as `https://apple.com:443/@msdn.com` and directs browsers to `apple.com`. In contrast, other camera and scanner apps preserve the original encoded sequence, directing browsers to `msdn.com`.

Furthermore, we found that support for Byte Order Mark (BOM) identification is also not universal across all browsers and QR code readers. BOM is a Unicode character used to signify the endianness of a text stream, which is particularly useful when dealing with text encoded in UTF-16 or UTF-32. By identifying the BOM, an app can determine how to correctly interpret and render the text. However, we found that none of the QR code scanners tested actually support this. On the browser front, BOM support is slightly better. For instance, when we utilize the UTF-16LE BOM and encode our payload string in UTF-16LE format before embedding it into a QR code, we observe that several browsers, namely Chrome on Android, Edge on Android, and Chrome on iOS, can successfully identify the BOM sequence and produce the expected output. The other browsers fail to detect the BOM and thus receive an unintelligible text.

**Table 5: Inter-App Handling on iOS: Newline URL Payload**

| **Payload Data** | `https://apple.com\nmsdn.com` |
|---|---|
| *Expected Result* | `apple.commsdn.com` |
| Brave† (visit) | `apple.commsdn.com` |
| Chrome (display) | nothing |
| Chrome (visit) | $\mathcal{S}^*$ |
| Edge‡ (visit) | `apple.commsdn.com` |
| Firefox (display) | `apple.com`<br>`msdn.com` |
| Firefox (visit) | `apple.com%0Amsdn.com` |
| Opera‡ (visit) | $\mathcal{S}^*$ |
| UC Browser‡ (visit) | $\mathcal{S}^*$ |
| Vivaldi† (visit) | `apple.commsdn.com` |
| iOS Camera (display) | `com`<br>`msdn.com` |
| iOS Safari (visit) | `apple.com%0Amsdn.com` |

$\mathcal{S}^*$: Searches for the decoded content using the default search engine.
†: After scanning a QR code, these browsers populate the address bar with the URL and let the user decide whether to visit it.
‡: After scanning a QR code, these browsers navigate directly to the link without displaying a prompt.

**Table 6: Inter-App Handling on iOS: Carriage-Return URL Payload**

| **Payload Data** | `apple.com\rmsdn.com` |
|---|---|
| *Expected Result* | `apple.commsdn.com` |
| Brave† (visit) | `apple.commsdn.com` |
| Chrome (display) | nothing |
| Chrome (visit) | $\mathcal{S}^*$ |
| Edge‡ (visit) | $\mathcal{S}^*$ |
| Firefox (display) | `apple.com`<br>`msdn.com` |
| Firefox (visit) | `apple.com%0Dmsdn.com` |
| Opera‡ (visit) | $\mathcal{S}^*$ |
| UC Browser‡ (visit) | `apple.com%0Dmsdn.com` |
| Vivaldi† (visit) | `apple.commsdn.com` |
| iOS Camera (display) | `apple.com`<br>`msdn.com` |
| iOS Safari (visit) | `apple.com` |

$\mathcal{S}^*$: Searches for the decoded content using the default search engine.
†: After scanning a QR code, these browsers populate the address bar with the URL and let the user decide whether to visit it.
‡: After scanning a QR code, these browsers navigate directly to the link without displaying a prompt.

## 5.3 Inter-OS Discrepancies

We now discuss the variations in QR code payload processing that exist across different OSes. We compare the native Camera apps on these platforms because users might use them to scan QR codes. We also compare whether the same browser behaves differently on different OSes.

From the findings discussed above, we can already see differences between the Android and iOS platforms. For example, Android apps have more consistencies in handling backslashes than

iOS. Furthermore, on the Android operating system, if a QR code payload contains a null character represented as `\x00`, the system tends to ignore this particular character altogether. This means that the presence of `\x00` does not affect the overall content that is extracted from the QR code, allowing the rest of the string to be processed and displayed as intended. In contrast, the behavior of the iOS system regarding the same payloads differs significantly. When an iOS application encounters a QR code payload string that includes a null character (`\x00`), it interprets this character as an indication of termination. Consequently, the iOS system will terminate the string at the point of the null character, leading to a potentially incomplete or erroneous interpretation of the QR code's intended content. Knowing this, one can craft a QR code payload that will be interpreted differently on different OSes. For instance, with the payload string `https://apple.com\x00@msdn.com`, all iOS applications will direct users to `apple.com`, while the majority of Android apps, except for Opera, will direct users to `msdn.com`.

When it comes to special characters, such as zero-width spaces and others, we observe that applications across different operating systems also lack consistency in their behavior. Of all the native camera apps, we found that only Samsung Camera app supports the overlong UTF-8 encoding. When a UTF-8 character is encoded in an overlong format, no other apps can successfully parse a QR code containing such characters. However, Samsung Camera can correctly decode the payload contents. For example, when a normal forward slash is overlong-encoded into a 2-byte representation as `\xc0\xaf`, a QR code containing the payload `http://apple.com\xc0\xafmac` would typically fail to parse in other apps. In this instance, Samsung Camera successfully decodes the QR code and directs the user to `http://apple.com/mac`.

Furthermore, on iOS, we found that HTTP prefixes and TLDs play an important role in domain name recognition. In our testing, we used two domain names: one with an uncommon TLD (`www.citi`) and one well-known domain (`apple.com`), inserting an ASCII character between them. When this character is not a valid domain character, iOS separately evaluates both domains and prioritizes the one it recognizes, as shown in Table 7. This creates an interesting phenomenon where even when the character has actual meaning in URLs (such as `/`, ?, or #), iOS still selects `apple.com`. For example, given the payload `www.citi/apple.com`, iOS resolves to `www.citi` but Android resolves to `www.citi/apple.com`. When an HTTP prefix is added, iOS prioritizes parsing the domain with the HTTP prefix. Similarly, when encountering characters like `/`, ?, or #, iOS still prioritizes the part with the HTTP prefix while ignoring other parts. This results in iOS resolving to apple.com when the payload is `www.citi#http://apple.com`, while Android resolves to `www.citi`.

It is also worth noting that the same application can demonstrate slight variations in functionality across different OSes. For instance, the Brave browser on Android tends to utilize Punycode to represent IDN domains in its address bar, while on iOS, it renders the corresponding characters directly, which potentially exposes users to visual deceits. Similarly, Opera on Android disregards certain characters; for example, when presented with the payload `https://apple.com\xC4\x80msdn.com`, Opera on Android directly accesses `apple.commsdn.com`, whereas Opera on iOS resorts to a search engine for resolution. Furthermore, Edge on Android employs pop-up prompts, while on iOS, it directly navigates to the URL if the QR code content contains a recognizable URL, a more permissive behavior that benefits phishing attacks. Additionally, when encountering `apple\uff61com`, where `\uff61` represents a half-width ideographic stop, the iOS version of Edge opens up `apple.com`, while the Android version defaults to the search engine, highlighting yet another inconsistency in QR code processing across OSes.

**Table 7: Examples of Inter-OS Discrepancies**

| **Payload Data** | www.citi[$S$]apple.com | apple.com[$S$]www.citi |
|---|---|---|
| **iOS** | apple.com | apple.com |
| **Android** | www.citi{$S$}apple.com | apple.com{$S$}www.citi |
| **Payload Data** | http://www.citi[$S$]apple.com | www.citi[$S$]http://apple.com |
| **iOS** | http://www.citi{$S$}apple.com | apple.com |
| **Android** | http://www.citi{$S$}apple.com | www.citi{$S$}http://apple.com |

[$S$]: The raw injected special character (*e.g.*, `\x00`).
{$S$}: The same special character escaped or encoded.

## 5.4 Other Findings

In addition to logical discrepancies, we also discovered that certain apps can crash after scanning QR codes with special payloads. For example, Edge and Trend Micro's QR Scanner on iOS, as well as Vivaldi on Android, crash when processing payload strings that include overlong UTF-8 encoding. In addition, BOM markers can cause crashes in Vivaldi on Android and Kaspersky's QR Scanner on iOS. We also found that QR codes containing URLs with the `file://` scheme cause the Samsung Camera app to crash. We investigated the root causes of Android crashes with `adb`. In Vivaldi, scanning a QR code containing an overlong UTF-8 URL or a BOM marker throws `java.lang.NullPointerException` in `java.lang.String.hasCode()`. Samsung Camera does not correctly handle `file://` URIs. The crash occurs when the camera app shares a `file://` URI with another app, which Android has prohibited since version 7.0 (API level 24). As such, these crashes are unlikely to have other exploits.

# 6 Implications and Root Cause Analysis

## 6.1 Real-World Exploitation

The in-app discrepancies (Section 5.1) show that quite a number of browsers and QR code readers are inadequate in helping users decide whether the destination URL is genuine or not. As such, users could be deceived by a purposely crafted QR code payload in a targeted phishing attack. Similarly, knowing the inter-app discrepancies (Section 5.2), attackers can craft QR code payloads that appear benign to users of certain apps, but will actually open up a malicious Website for users of other apps. This can help make a targeted phishing attack more difficult to get noticed.

We also found the inter-OS discrepancies (Section 5.3) to have security implications beyond loading malicious Websites in browsers. Specifically, we noticed that many peer-to-peer payment apps are designed to handle payment links of their proprietary domains. With this, the payment sender can launch a monetary transaction by scanning the QR code of the intended recipient using the system camera app. After decoding and parsing the QR code payload, the recipient's payment link will be passed to the sender's payment app, and the sender can then specify the amount of the transaction. Knowing that the camera apps of iOS, stock Android and branded

Android have different parsing preferences, it is possible for an attacker to craft a payment QR code that, when scanned by users of different OSes, sends money to different recipients. As an example, using the system camera app to scan a QR code with payload `paypal.com/Alice\rEve`, iOS users will get Alice as the PayPal recipient, but Android users will get AliceEve instead. Likewise, we found that one of the banks in our jurisdiction runs a peer-to-peer payment service, with payment recipient links on its proprietary domain `payme.hsbc`. Thus, one can create a QR code containing the payload `payme.hsbc/Alice/?https://payme.hsbc/Eve`, and users of Apple Camera will get Eve as the transaction recipient, but Android users will get Alice instead. And since this payment service allows users to choose their own profile names and pictures, Eve can make her profile appear indistinguishable from that of Alice.

This is particularly worrisome, as brick-and-mortar stores increasingly accept electronic payments, and they sometimes use a *third-party printing service* (e.g., Walmart) to print business cards and banners containing their payment receiving QR code. Leveraging these processing discrepancies, a malicious printing service can then craft a malicious QR code that sends payment to the original store when scanned by users of one camera app (e.g., Google Lens on Android), but for users of a different camera app (e.g., Apple Camera on iOS), sends payment to the attacker. Depending on payment platforms, the attacker might be able to set up a recipient profile that appears indistinguishable from that of the original store (e.g., similar recipient name and profile picture), further improving the covertness of the attack.

To gauge the generalizability of this attack, we performed a Google Search with the keywords including "P2P payment link", "URL", "QR Code", "payment app", and "money transfer", to discover more payment services that might be susceptible to this threat. For apps that we can download and install, we create an account and get the payment link. For others, we look for their documentation and advertisements to get sample links and QR codes. Through this investigation, we compiled a list of payment apps that are potentially exploitable due to having a similarly structured payment link, including services that are popular in major economies. Table 8 shows the list of potentially exploitable payment apps and their payment link formats.

During our investigation, we found that these payment services typically allow users considerable flexibility in selecting profile pictures, while imposing stricter controls on display names. The specific constraints vary across platforms. For instance, the payment application available in our jurisdiction permits free selection of both names and profile pictures. Based on our prior experience, Venmo allows users to freely choose profile pictures and permits display name changes with moderate frequency restrictions. PayPal enables profile picture customization but may require supporting documents for name changes. We are unable to conduct hands-on testing of all identified payment applications due to jurisdictional restrictions on obtaining the necessary legal identification documents required for account registration.

**Table 8: Candidate Payment-Link Formats**

| Payment Application | Payment Links |
|---|---|
| ANNA Money | pay.anna.money/[USERNAME] |
| Cardlink | cardlink.gr/[USERNAME] |
| Cash App | cash.app/[USERNAME] |
| Geidea | www.geidea.net/[USERNAME] |
| Lazerpay | lazerpay.finance/pay/[USERNAME] |
| Mercury | mercury.com/pay/[USERNAME] |
| Monzo | monzo.me/[USERNAME] |
| PayMe | payme.hsbc/[USERNAME] |
| PayPal | paypal.com/paypalme/[USERNAME] |
| Venmo | account.venmo.com/u/[USERNAME] |

## 6.2 Root Cause Analysis

**In-App discrepancies.** The exploitable in-app discrepancies generally stem from developers failing to give adequate attention to Internationalized Domain Name (IDN) in payload URLs. Many developers tend to perform only basic, general checks on URLs, overlooking the visual confusions introduced by IDN domains. Furthermore, the use of UI elements that are capable of rendering Unicode text also contributes to these issues.

For example, after investigating the source code of the Firefox Android app, we found that the `URLStringUtils.toNormalizedURL` function is employed to process URLs before they are shown in the pop-up prompt. However, this function primarily focuses on standard URL normalization through the `android.net.Uri.parse` method. Notably, it does not convert IDN domains into Punycode, which encodes Unicode characters in ASCII representation. With the return value of `toNormalizedUrl` being directly used in the pop-up prompt and passed to `openToBrowserAndLoad` in `HomeActivity` to load the URL, the URL displayed to the user may not be the same as the one that is actually loaded by Firefox. This oversight explains the gap between display shown to the user and the actual URL that gets loaded. The related code can be found at [38] and [39].

On the other hand, our analysis of Trend Micro Secure QR Scanner shows that it employs a cloud-based service to determine the safety of URLs. Without access to its source code, we speculate that the vulnerability discussed in Section 5.1 is caused by the inability of its cloud service to correctly interpret optional credentials in the authority part of a URL, and thus it gets tricked by the `@` symbol.

**Inter-App discrepancies.** In this work, we observed that many scanners and browsers do not actually follow the WHATWG specification in how to handle and interpret URLs, resulting in many interesting discrepancies. Nevertheless, an in-depth analysis of the browsers' internal implementations that explains the observed UI behavior remains challenging. Although Opera, Vivaldi and Edge are all built on the Chromium framework, their UI components—including the QR code processing logic—are developed independently and remain proprietary. Because these components are not fully open-source [35], [49], we resort to an API-level comparison to gain insights on the root causes.

We noticed that the handling of URLs in Firefox, Brave and Opera follow different methodologies. In Firefox, the application extracts raw QR code data using the Google ZXing library, processes the retrieved content through `android.net.Uri.parse`, and subsequently passes it to the browser engine to load the URL. This process involves validation checks to ensure the URL conforms to standard formatting rules. In contrast, Brave utilizes the `com.google.android.gms.vision.barcode.Barcode` library, relying on

the `barcode.displayValue` method provided by the library to obtain the QR code value, which is then loaded by the browser engine. However, as noted in the gms-vision documentation [15], `displayValue` is not guaranteed to match the raw data embedded in the QR code. This value is only available when the payload is encoded in UTF-8 format; for non-UTF-8 encoded payloads, the documentation recommends using `getRawBytes` instead to ensure accurate data retrieval. Opera also utilizes gms-vision for QR code decoding. However, analysis of raw QR code data collected via Frida scripts reveals that Opera truncates content after encountering a null byte `\x00`. This indicates that, despite using the same API as Brave, Opera's developers implemented additional processing, which explains some of the observed inconsistencies. Opera's source-level root causes are difficult to pinpoint because of its obfuscation; however, the relevant code of Brave can be found at [7].

**Inter-OS discrepancies.** Inter-OS discrepancies mainly stem from the fact that the underlying parsing and string normalization logic behind the system APIs are different. For instance, `Uri.parse` is a standard API method provided by the official Google Android library `android.net.Uri`. When a backslash is present in the string (e.g., `https://apple.com\@msdn.com`), the API method interprets it as a slash and parses the subsequent portion as a path component. In contrast, on iOS, whether using `URL` in Swift or `NSURLComponents` and `NSURL` in Objective-C, such URLs are consistently treated as invalid. This discrepancy may compel developers to either adopt the more limiting URL formats or implement custom parsing logic, leading to inconsistent behaviors across different applications, especially when it comes to processing URLs containing special characters.

Furthermore, the iOS API provides the `AVMetadataMachineReadableCodeObject` class for processing barcodes and QR codes. We developed a small demo application based on this class to extract raw QR code data. During our testing, we encountered a critical issue: if no specific error handling is implemented, the demo application crashes when the scanned string contains overlong UTF-8 sequences or a Byte Order Mark. This behavior mirrors and might explain the crash observed in Edge on iOS discussed in Section 5.4.

## 7 Discussions

**Limitations.** In our experiments, we mainly focused on URL payloads, which has a well-defined structure and is commonly used in many application scenarios. QR code can also encode payload data in other formats, including binary data. We leave the testing of these lesser-known payload formats for future work.

In this study, we excluded retail and e-commerce applications, some of which might also have QR code scanning functionalities. That said, many tend to only accept specific domains owned by their corresponding companies, which limits their exposure to the URL processing discrepancies we investigate. We leave for future work the investigation of whether such apps can also serve as entry points of potentially malicious content.

For testing Android apps, we have a semi-automated framework, which employs AVD Snapshot to accelerate testing. While in theory the same design can also work on iOS, implementing it is challenging due to the restrictive nature of the iOS ecosystem, particularly regarding dynamic binary instrumentation. Unlike Android, where tools for intent interception and hook-based monitoring are mature, iOS imposes stringent sandboxing and code-signing mechanisms that make dynamic instrumentation much harder. Although jailbreaking could be a potential solution, implementing a reliable jailbreak for our purpose is also challenging. Our iOS device has an A13 SoC with iOS version 17.5.1. Although jailbreak solutions exist for A13 devices, they are typically limited to iOS versions 16.5.1 and below. Achieving a jailbreak with the necessary system-level access on A13 devices running iOS 17 or later is currently not feasible with known available methods [17, 48]. Researchers who have subscription to virtualized environments such as Corellium [9] might fare better. We also acknowledge that it might be possible to increase the level of automation by building a similar setup used in a recent work [8]. Nevertheless, since we have a targeted set of deceptive inputs, the manual effort is manageable. Therefore, in this work, we relied on manual testing for iOS apps. It took two of our team members a total of 32 man-hours to finish all the iOS testing.

Finally, because of the difficulties in creating online banking accounts in other countries, we are unable to test a large variety of payment apps. Nevertheless, researchers in other jurisdictions can build on our findings to test banking apps that they can access.

**Responsible Disclosure.** For the in-app discrepancies (Section 5.1), we have already reported to the maintainers of Sony camera, Samsung camera, Edge, Chrome and Firefox. Google has confirmed the problems found in Chrome and rewarded us with a bug bounty. Discussions with several vendors are ongoing. Some said the observed behaviors are intended, and maintainers of the local payment app (Section 6.1) said this is a problem with the OS APIs and not a bug in their products. A detailed account with several vendors can be found below. In short, there is a lack of incentives to bring a unified user experience across different vendors.

**Google.** Three Chrome (iOS) vulnerabilities were confirmed as medium severity by Chromium, though fixes were delayed due to initial debates over whether issues stemmed from Google Lens or Chrome. In the end, our reports were resolved with bounty rewards and a pending CVE. A separate Pixel Camera app report was dismissed by another Google security team as a social engineering risk and not tracked.

**Samsung.** Three reports were rejected as insufficiently severe, reclassified as feature enhancement requests. Samsung argued QR scanning is a pre-browser step and redirected reports to its development team for potential future updates.

**Microsoft.** Declined fixes, citing intentional design choices and the problems require social engineering/user interaction to exploit. Notably, this has led to an interesting divergence in Edge's user experience across mobile platforms: Android has post-scan confirmation pop-ups, while iOS retains direct URL opening.

**Apple.** We have reported the subtle TLD parsing issue and the discrepancy in decoding special characters on iOS. They marked our findings as expected behavior.

**Firefox.** We submitted two reports regarding the in-app discrepancies and potential exploits. They were marked as duplicates of another report [40], which remains unfixed to this day.

**Secure QR Scanners.** We also submitted reports to Trend Micro and Kaspersky; however, we have yet to receive any responses from either company.

For inter-OS discrepancies (Section 5.3) and specifically the payment service in our local market (Section 6.1), we have reported our findings to Google, Apple, the bank who operates the payment service, as well as the banking authority in our jurisdiction. The latter two consider this issue to be outside their authority. The app maintainers at the bank mentioned that the discrepancies are caused by the OS APIs, not a bug in their app code. Apple responded to our report saying that the backward scanning is an expected behavior, and they have no plans of changing it. We also shared the other inter-app discrepancies (Section 5.2) with maintainers of browsers and QR code readers. We open-source release our code and test cases at https://github.com/ZeddYu/Chimera_QRCode to help the community better address these discrepancies.

**Mitigation.** We outline a procedure for processing URL payloads decoded from QR codes. First, an application decodes the payload using a specified character encoding and rejects invalid byte sequences, NULL bytes, and control characters that receive different interpretations in the tested implementations. Second, it parses the resulting string with a specified URL parser that follows the WHATWG URL Standard and produces a normalized URL. URL safety checks and the confirmation interface should use this same normalized URL, with the complete host displayed to the user. Finally, after user confirmation, the application should pass the same normalized URL string to the browser or target application. In this way, display, safety checking, and execution use the same processed URL.

For handling IDN domains, we recommend developers to implement the IDN algorithm described by Chrome[20], which documents which combinations of characters/numerals require showing as punycode. This approach ensures that non-ASCII characters are accurately represented in a format that is universally understood by systems, and do not cause visual confusions. When it is necessary to render and display IDN domains, one can consider showing both the original IDN domains as well as their corresponding Punycode. This provides sufficient information to aid the user in identifying potential phishing URLs. For URL parameters and query strings that might contain Unicode characters, we advocate the use of URL encoding whenever feasible. This can enhance compatibility and prevent misinterpretation of URLs.

For inter-app and inter-OS, it seems that clarifications from standard specification are necessary to unify the behaviors across apps and platforms. While the QR code standard document [1] is very detailed on how to recognize and decode the QR code image, to the best of our knowledge, how to parse and process the URL payloads is currently not standardized. As such, developers are free to implement their own interpretations, and this partly explains why some believe their code is having "expected behaviors". To be specific, we suggest that upon retrieving a URL from a QR Code, it should be processed in accordance with the WHATWG standard to establish a unified outcome; however, this requires consensus from the community.

## 8 Related Work

**QR Code Security and Privacy.** Previous studies have investigated the security and privacy threats associated with QR codes. Krombholz *et al.*[28] conducted a survey of possible attacks and usability challenges, including QR code as a possible vehicle for delivering phishing links. Kieseberg *et al.*[27] demonstrated that minor alterations to pixels of a QR code can change the payload content that it delivers. Several studies then investigated the effectiveness of QR code scanners [57, 58, 63] in detecting phishing Websites. On the other hand, Han *et al.*[23] focused on the security of Resource Access Handlers that are triggered by QR code scanning, revealing that many applications lack adequate restrictions on RAH calls, exposing users to considerable vulnerabilities. The most relevant work is perhaps [8], in which researchers proposed a toolkit for QR code fuzzing, with a focus on the robustness of QR code decoders (stage 1 of the processing pipeline).

Existing work on QR code security testing mainly focused on vulnerabilities in stage 1 (QR code decoding) and stage 3 (final payload processing). Our work addresses this research gap by systematically exploring vulnerabilities in stage 2 (payload parsing and display) and the transition from stage 2 to 3. We identify In-App and Inter-App discrepancies during the payload parsing and display process. For instance, the display truncation vulnerabilities are specific to the stage 2 user prompt interface and have not been explored in previous studies. Furthermore, we investigate Inter-OS discrepancies that lead to different security outcomes across platforms.

Prior work on browser vulnerabilities [10–14] has documented some in-app discrepancies within web browsers. However, our work extends this understanding by revealing new attack surfaces that emerge from interactions between different apps and platforms. We also demonstrate practical exploits that leverage these in-app, inter-app, and inter-OS discrepancies.

**Discrepancy and Attacks.** Attacks caused by discrepancies in competing implementations have been studied in many different contexts. For instance, several efforts went into testing inconsistencies in HTTP header parsing [24, 25, 54], which is known to be a source of attacks. Likewise, URL parsers have also been scrutinized by recent efforts [2, 50, 55], where discrepancies can lead to request forgery and confusion attacks. Beyond Web technologies, researchers have also investigated bugs that cause inconsistent document rendering in PDF readers [29], as well as the discrepancies in rendering and digital signature computation of PDF [37, 53], OpenDocument [51] and Microsoft Office documents [52], which can enable various deceptive attacks. Similar to our study, some of these efforts also used targeted testing instead of large-scale fuzzing to investigate display rendering issues.

## 9 Conclusion

This paper systematically analyzes and explores discrepancies in how popular mobile apps handle QR code payloads. We categorize these into in-app, inter-app, and inter-OS discrepancies and discuss potential exploits. To effectively discover such discrepancies, we compiled a collection of 126 representative test cases. An automation framework was designed and built for testing on Android. Through systematic testing, we discovered many in-app, inter-app, and inter-OS discrepancies that might be utilized in potential attacks. We also discuss their root causes and possible mitigations. Our results suggest that much remains to be done to provide users of different apps and devices with a uniform QR code scanning experience.

## 10 Acknowledgements

We thank the anonymous reviewers for their constructive feedback. This work was supported in part by a grant from the Research Grants Council (RGC) of Hong Kong (Project No.: CUHK 24205021), and research grants from the Department of Information Engineering at the Chinese University of Hong Kong (project code: NEW/SYC, GRF/23/SYC, and GRF/24/SYC). We used Claude (Anthropic) solely for checking grammar and spelling.

## 11 Ethics Considerations

The QR codes prepared were used only in our experiments, and all QR code scanning was conducted within a controlled environment. All tested apps were run on devices that we own, and we never launched the Chimera attack against any other users. We used only our own accounts for the proof-of-concept attack against payment services (Section 6.1) and did not send any monetary transactions to other recipients.

## 12 Artifact Availability

The setup and test cases of our investigation are available at https://github.com/ZeddYu/Chimera_QRCode. All QR code images used in this work are available at https://github.com/ZeddYu/Chimera_QRCode/fuzzing_imgs/fuzzing_imgs.7z.

## References

[1] ISO/IEC 18004:2015. 2015. ISO/IEC 18004:2015. https://www.iso.org/standard/62021.html. [Accessed 28-08-2024].
[2] Dashmeet Kaur Ajmani, Igibek Koishybayev, and Alexandros Kapravelos. 2022. yoU aRe a Liar://A Unified Framework for Cross-Testing URL Parsers. In *2022 IEEE Security and Privacy Workshops (SPW)*. 51–58. doi:10.1109/SPW54247.2022.9833883
[3] Tim Berners-Lee, Roy Fielding, and Larry Masinter. 2005. RFC 3986: Uniform resource identifier (uri): Generic syntax.
[4] Jonathan Birch. 2019. Host/Split: Exploitable Antipatterns in Unicode Normalization. *Black Hat USA 2019* (2019).
[5] Nicholas Boucher, Luca Pajola, Ilia Shumailov, Ross Anderson, and Mauro Conti. 2023. Boosting Big Brother: Attacking Search Engines with Encodings *(RAID '23)*. Association for Computing Machinery, New York, NY, USA, 700–713. doi:10.1145/3607199.3607220
[6] Nicholas Boucher, Ilia Shumailov, Ross Anderson, and Nicolas Papernot. 2022. Bad characters: Imperceptible nlp attacks. In *2022 IEEE symposium on security and privacy (SP)*. IEEE, 1987–2004.
[7] Brave. 2025. brave/brave-core. https://github.com/brave/brave-core/blob/c52b3fac1a1ab39da360cd31d48ca1b1c45b4c78/android/java/org/chromium/chrome/browser/omnibox/BraveLocationBarQRDialogFragment.java.
[8] Federico Carboni, Mauro Conti, Denis Donadel, and Mariano Sciacco. 2023. If You're Scanning This, It's Too Late! A QR Code-Based Fuzzing Methodology to Identify Input Vulnerabilities in Mobile Apps. In *International Conference on Applied Cryptography and Network Security*. Springer, 553–570.
[9] Corellium. 2025. Corellium Platform | Research, Work, Test Arm-Based Devices. https://www.corellium.com/platform.
[10] CVE-2000-0884. 2000. CVE-2000-0884. https://nvd.nist.gov/vuln/detail/CVE-2000-0884.
[11] CVE-2017-5060. 2017. CVE-2017-5060. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-5060.
[12] CVE-2018-4205. 2018. CVE-2018-4205. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2018-4205.
[13] CVE-2023-28364. 2023. Open redirect due to scanning QR code via brave browser — hackerone.com. https://hackerone.com/reports/1946534.
[14] CVE-2024-0953. 2024. 1837916 - (CVE-2024-0953) Firefox for iOS QR Code Scanner does not show the URL for user confirmation before opening it — bugzilla.mozilla.org. https://bugzilla.mozilla.org/show_bug.cgi?id=1837916.
[15] Google developers. 2025. Barcode |ML Kit | Google for Developers — developers.google.com. https://developers.google.com/android/reference/com/google/mlkit/vision/barcode/common/Barcode.
[16] McAuliff et al. 2025. FAQ | iOS Guide. https://ios.cfw.guide/faq/#what-is-the-latest-ios-version-with-a-jailbreak-for-my-64-bit-device.
[17] McAuliff et al. 2025. FAQ | iOS Guide. https://ios.cfw.guide/faq/.
[18] Sidong Feng and Chunyang Chen. 2024. Prompting is all you need: Automated android bug replay with large language models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
[19] Google. 2022. Pixel Camera: QR code decoder error with .at domains. https://issuetracker.google.com/issues/214884571.
[20] Google. 2025. Internationalized Domain Names (IDN) in Google Chrome. https://chromium.googlesource.com/chromium/src.git/+/master/docs/idn.md.
[21] HackerNews. 2022. Google Camera randomly changes some QR code URLs on Android 12. https://news.ycombinator.com/item?id=30015810.
[22] Hafiizh. 2021. 1736886 - (CVE-2021-43530) uxss on qrcode code reader (mozilla android version: 93.2.0) — bugzilla.mozilla.org. https://bugzilla.mozilla.org/show_bug.cgi?id=1736886. [Accessed 04-09-2024].
[23] Xing Han, Yuheng Zhang, Xue Zhang, Zeyuan Chen, Mingzhe Wang, Yiwei Zhang, Siqi Ma, Yu Yu, Elisa Bertino, and Juanru Li. 2023. Medusa attack: exploring security hazards of in-app QR code scanning. In *Proceedings of the 32nd USENIX Conference on Security Symposium* (Anaheim, CA, USA) *(SEC '23)*. USENIX Association, USA, Article 258, 18 pages.
[24] Bahruz Jabiyev, Steven Sprecher, Anthony Gavazzi, Tommaso Innocenti, Kaan Onarlioglu, and Engin Kirda. 2022. {FRAMESHIFTER}: Security implications of {HTTP/2-to-HTTP/1} conversion anomalies. In *31st USENIX Security Symposium (USENIX Security 22)*. 1061–1075.
[25] Bahruz Jabiyev, Steven Sprecher, Kaan Onarlioglu, and Engin Kirda. 2021. T-reqs: Http request smuggling with differential fuzzing. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 1805–1820.
[26] Dylan Katz. 2019. 1590288 - Design flaws in the QR scanner and URL bar leads to QR phishing vector for executing javascript — bugzilla.mozilla.org. https://bugzilla.mozilla.org/show_bug.cgi?id=1590288. [Accessed 04-09-2024].
[27] Peter Kieseberg, Sebastian Schrittwieser, Manuel Leithner, Martin Mulazzani, Edgar Weippl, Lindsay Munroe, and Mayank Sinha. 2012. Malicious pixels using QR codes as attack vector. *Trustworthy ubiquitous computing* (2012), 21–38.
[28] Katharina Krombholz, Peter Frühwirt, Peter Kieseberg, Ioannis Kapsalis, Markus Huber, and Edgar Weippl. 2014. QR code security: A survey of attacks and challenges for usable security. In *Human Aspects of Information Security, Privacy, and Trust: Second International Conference, HAS 2014, Held as Part of HCI International 2014, Heraklion, Crete, Greece, June 22-27, 2014. Proceedings 2*. Springer, 79–90.
[29] Tomasz Kuchta, Thibaud Lutellier, Edmund Wong, Lin Tan, and Cristian Cadar. 2018. On the correctness of electronic documents: studying, finding, and localizing inconsistency bugs in PDF readers and files. *Empirical Software Engineering* 23 (2018), 3187–3220.
[30] Markus Kuhn. 2024. UTF-8 decoder capability and stress test. https://www.cl.cam.ac.uk/~mgk25/ucs/examples/UTF-8-test.txt.
[31] Jungwon Lim, Yonghwi Jin, Mansour Alharthi, Xiaokuan Zhang, Jinho Jung, Rajat Gupta, Kuilin Li, Daehee Jang, and Taesoo Kim. 2021. SOK: On the Analysis of Web Browser Security. *ArXiv* abs/2112.15561 (2021). https://api.semanticscholar.org/CorpusID:245634339
[32] Baojun Liu, Chaoyi Lu, Zhou Li, Ying Liu, Hai-Xin Duan, Shuang Hao, and Zaifeng Zhang. 2018. A Reexamination of Internationalized Domain Names: The Good, the Bad and the Ugly.. In *DSN*. 654–665.
[33] Markus Scherer Mark Davis. 2024. Unicode IDNA Compatibility Processing. https://www.unicode.org/reports/tr46/. [Accessed 30-10-2024].
[34] Michel Suignard Mark Davis. 2024. Unicode Security Considerations. https://www.unicode.org/reports/tr36/tr36-15.html. [Accessed 30-08-2024].
[35] MicrosoftEdge. 2025. MSEdge. https://github.com/MicrosoftEdge/MSEdge.
[36] mitmproxy. 2024. mitmproxy. https://github.com/mitmproxy/mitmproxy. Accessed: 2024-08-29.
[37] Vladislav Mladenov, Christian Mainka, Karsten Meyer zu Selhausen, Martin Grothe, and Jörg Schwenk. 2019. 1 trillion dollar refund: How to spoof pdf signatures. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 1–14.
[38] Mozilla. 2025. mozilla-central@ hg-edge.mozilla.org. https://hg-edge.mozilla.org/mozilla-central/file/tip/mobile/android/fenix/app/src/main/java/org/mozilla/fenix/search/SearchDialogFragment.kt.
[39] Mozilla. 2025. mozilla-central@ hg-edge.mozilla.org. https://hg-edge.mozilla.org/mozilla-central/file/tip/mobile/android/android-components/components/support/ktx/src/main/java/mozilla/components/support/ktx/kotlin/String.kt.
[40] mozilla. 2025. QR code scanner does not canonicalize URLs the same way as GeckoView (e.g. IDN), potential for spoofing. https://bugzilla.mozilla.org/show_bug.cgi?id=1691472.
[41] mrzheev. 2023. UXss on brave browser via scan QR Code. https://hackerone.com/reports/1884042. [Accessed 04-09-2024].
[42] newbit. 2024. rootAVD. https://gitlab.com/newbit/rootAVD. Accessed: 2024-08-29.
[43] Nikhil. 2019. 1567118 - (CVE-2019-17003) Improper parsing of QR codes in address bar leads to XSS — bugzilla.mozilla.org. https://bugzilla.mozilla.org/show_bug.cgi?id=CVE-2019-17003. [Accessed 04-09-2024].
[44] oleavr. 2025. Frida, A world-class dynamic instrumentation toolkit. https://frida.re/

[45] owasp. 2025. XSS. https://owasp.org/www-community/attacks/xss/.
[46] PaddlePaddle. 2025. PaddlePaddle/PaddleOCR. https://github.com/PaddlePaddle/PaddleOCR.
[47] Wladimir Palant. 2022. 1705094 - (CVE-2022-22749) QR code scanner allows opening non-HTTP URLs — bugzilla.mozilla.org. https://bugzilla.mozilla.org/show_bug.cgi?id=1705094. [Accessed 04-09-2024].
[48] palera1n. [n. d.]. palera1n. https://github.com/palera1n/palera1n.
[49] Julien Picalausa. 2021. Why isn't Vivaldi browser open-source? | Vivaldi Browser — vivaldi.com. https://vivaldi.com/blog/technology/why-isnt-vivaldi-browser-open-source/.
[50] Joshua Reynolds, Adam Bates, and Michael Bailey. 2022. Equivocal URLs: Understanding the Fragmented Space of URL Parser Implementations. In *Computer Security – ESORICS 2022: 27th European Symposium on Research in Computer Security, Copenhagen, Denmark, September 26–30, 2022, Proceedings, Part III* (Copenhagen, Denmark). Springer-Verlag, Berlin, Heidelberg, 166–185. doi:10.1007/978-3-031-17143-7_9
[51] Simon Rohlmann, Christian Mainka, Vladislav Mladenov, and Jörg Schwenk. 2022. Oops... Code Execution and Content Spoofing: The First Comprehensive Analysis of OpenDocument Signatures. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 3075–3092. https://www.usenix.org/conference/usenixsecurity22/presentation/rohlmann
[52] Simon Rohlmann, Vladislav Mladenov, Christian Mainka, Daniel Hirschberger, and Jörg Schwenk. 2023. Every Signature is Broken: On the Insecurity of Microsoft Office's OOXML Signatures. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 7411–7428. https://www.usenix.org/conference/usenixsecurity23/presentation/rohlmann
[53] Simon Rohlmann, Vladislav Mladenov, Christian Mainka, and Jorg Schwenk. 2021. Breaking the specification: PDF certification. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1485–1501.
[54] Kaiwen Shen, Jianyu Lu, Yaru Yang, Jianjun Chen, Mingming Zhang, Haixin Duan, Jia Zhang, and Xiaofeng Zheng. 2022. Hdiff: A semi-automatic framework for discovering semantic gap attack in http implementations. In *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 1–13.
[55] Orange Tsai. 2017. A new era of ssrf-exploiting url parser in trending programming languages. *Black Hat USA* (2017).
[56] Manuel Vonau. 2022. Google Camera update fixes issue that randomly changed QR code URLs on Android 12. https://www.androidpolice.com/google-camera-randomly-changes-some-qr-code-urls-on-android-12/.
[57] Heider AM Wahsheh and Flaminia L Luccio. 2020. Security and privacy of QR code applications: a comprehensive study, general guidelines and solutions. *Information* 11, 4 (2020), 217.
[58] Heider AM Wahsheh, Flaminia L Luccio, et al. 2019. Evaluating Security, Privacy and Usability Features of QR Code Readers.. In *ICISSP*. 266–273.
[59] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2024. AutoDroid: LLM-powered Task Automation in Android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking* (Washington D.C., DC, USA) *(ACM MobiCom '24)*. Association for Computing Machinery, New York, NY, USA, 543–557. doi:10.1145/3636534.3649379
[60] WHATWG. 2025. URL Standard — url.spec.whatwg.org. https://url.spec.whatwg.org/. [Accessed 02-09-2024].
[61] Wikipedia. 2025. Cryptojacking. https://en.wikipedia.org/wiki/Cryptojacking.
[62] Wikipedia contributors. 2025. Reed–Solomon error correction - Wikipedia — en.wikipedia.org. [Accessed 03-09-2024]. https://en.wikipedia.org/wiki/Reed%E2%80%93Solomon_error_correction
[63] Huiping Yao and Dongwan Shin. 2013. Towards preventing qr code based attacks on android phone using security warnings. In *Proceedings of the 8th ACM SIGSAC symposium on Information, computer and communications security*. 341–346.
[64] Juyeon Yoon, Robert Feldt, and Shin Yoo. 2023. Autonomous Large Language Model Agents Enabling Intent-Driven Mobile GUI Testing. *CoRR* abs/2311.08649 (2023). arXiv:2311.08649 doi:10.48550/ARXIV.2311.08649